

Les pièges fréquents et comment les éviter

Contexte & Mémoire

Coller des logs bruts dans le prompt Donne tout le fichier à Claude au lieu d'isoler la partie pertinente. Le contexte se remplit en quelques échanges.

Ne pas utiliser CLAUDE.md Répéter les conventions du projet à chaque session. Centraliser dans `CLAUDE.md` tout ce qui doit être constant.

Ignorer les avertissements de contexte Utiliser `/context` pour inspecter capacité et pertinence. Compacter ou préparer une passation si nécessaire ; aucun pourcentage ne garantit une qualité de réponse donnée.

Prompting

Demandes vagues `Refactorise ce fichier` sans contexte → résultat imprévisible. Préciser : quoi, pourquoi, contraintes.

Itérer sans checkpoint Enchaîner 10 échanges sans valider. Vérifier le diff après chaque changement significatif.

Accepter sans relire `--dangerously-skip-permissions` sans comprendre ce que fait Claude. Toujours auto-accept dans un scope limité.

Configuration

CLAUDE.md trop long Mettre toute la doc projet dans `CLAUDE.md`. Le contexte au démarrage doit être concis (< 200 lignes).

Hooks sans tests Écrire un hook complexe sans le tester en isolation. Tester d'abord manuellement, puis l'activer.

Ignorer settings.local.json Ne pas gitignorer ce fichier → fuites de config personnelle dans le dépôt de l'équipe.

Sécurité

Prompt injection non gérée Laisser Claude traiter du contenu externe (emails, issues) sans garde-fous. Utiliser un mode restreint avec whitelist d'outils.

Secrets dans le contexte Passer des clés API dans le prompt ou les lire via `@fichier`. Utiliser des variables d'environnement.

--dangerously-skip-permissions sans sandbox Exécuter en bypass total sur une machine partagée ou en production. Réserver aux environnements éphémères.

Règles d'Or

Donner des instructions claires et courtes, puis laisser Claude travailler. Revenir valider par petits incréments plutôt que de tout laisser faire d'un coup. Documenter les décisions importantes dans `CLAUDE.md` pour les prochaines sessions.